

在 CKKS 密文上实现多项式的 FFT 式奇偶拆分与重构

第二版：问题、主结果、正确性定理与伪代码

目录

1	Problem	2
1.1	目标	2
1.2	为什么选多项式域路线而不是系数向量矩阵路线	3
2	Notation and CKKS Semantics	3
2.1	环与嵌入	3
2.2	密文语义记号	3
2.3	本文使用的同态算子	4
2.4	上标 $\uparrow$ 的含义	4
3	Main Result	4
3.1	核心恒等式	4
3.2	实现 $Y \mapsto -Y$ 的特定自同构	5
4	Correctness Theorems (No Proofs)	5
5	Algorithms	6
5.1	约定	6
5.2	正向拆分：保留奇分支前面的 X^k	6
5.3	正向拆分：去掉奇分支前面的 X^k	6
5.4	反向重构：输入保留 X^k 的奇分支	7
5.5	反向重构：输入为归一化奇分支	7
5.6	一个最小执行模板	7
6	Implementation Notes	8
6.1	总则	8
6.2	OpenFHE	8
6.3	Microsoft SEAL	9
6.4	Lattigo v5	10
7	Practical Checklist	11

目录	2
8 Conclusion	11

摘要

本文给出一份面向实现的第二版整理。我们固定

$$P(Y) = \sum_{j=0}^{2n-1} p_j Y^j \in \mathbb{R}[Y]/(Y^{2n} + 1),$$

并研究如何在 CKKS 密文上直接完成 FFT 式奇偶拆分：

$$P(Y) = P_{\text{even}}(Y^2) + Y P_{\text{odd}}(Y^2),$$

以及对应的反向重构。与第一版不同，本文不再重复代数证明，而是采用

Problem $\rightarrow$ Main Result $\rightarrow$ Correctness Theorem $\rightarrow$ Algorithm

的组织方式，给出可直接转译到程序中的伪代码。文末附上 OpenFHE、Microsoft SEAL 与 Lattigo v5 的 API 对应说明，并指出实现时必须特别注意的同构索引、plaintext 构造、level/scale 对齐等事项。

1 Problem

1.1 目标

固定一个 2 的幂 $N \geq 1$ ，以及一个满足 $n \mid N$ 且 $n < N$ 的 2 的幂。令

$$k = \frac{N}{2n} \in \mathbb{Z}_{>0}, \quad Y = X^k.$$

于是

$$Y^{2n} = X^{2nk} = X^N = -1.$$

给定多项式

$$P(Y) = \sum_{j=0}^{2n-1} p_j Y^j \in \mathbb{R}[Y]/(Y^{2n} + 1),$$

定义其 FFT 式奇偶分支为

$$P_{\text{even}}(T) = \sum_{i=0}^{n-1} p_{2i} T^i, \quad P_{\text{odd}}(T) = \sum_{i=0}^{n-1} p_{2i+1} T^i. \quad (1)$$

本文考虑如下两个问题：

P1. 正向拆分问题：给定一个 CKKS 密文 ct_P ，其语义上表示

$$P(X^k) \in \mathcal{R}_N^{\mathbb{R}} = \mathbb{R}[X]/(X^N + 1),$$

如何同态地产生两个分支密文，分别表示

$$P_{\text{even}}(X^{2k}) \quad \text{和} \quad P_{\text{odd}}(X^{2k})$$

(或者表示 $X^k P_{\text{odd}}(X^{2k})$ 的非归一化奇分支)？

P2. 反向重构问题：给定上述两个分支密文，如何同态地恢复出表示 $P(X^k)$ 的原始密文？

1.2 为什么选多项式域路线而不是系数向量矩阵路线

虽然在抽象线性代数层面，可以把

$$(p_0, p_1, \dots, p_{2n-1})^\top$$

视为一个长度 $2n$ 的系数向量，并用抽取矩阵 E, O 去得到偶、奇系数子向量；但在标准 CKKS 的高层接口中，最自然的对象是：

- 槽向量表示（经编码后的复向量）；
- 多项式明文表示（环中的多项式）。

如果走 E, O 的路线，通常需要先做一次 CoeffToSlot ，把系数信息显式搬到槽里；这会引入额外的线性变换成本。本文因此采用更直接的多项式域恒等式：

$$P(Y) = P_{\text{even}}(Y^2) + Y P_{\text{odd}}(Y^2), \quad (2)$$

$$P(-Y) = P_{\text{even}}(Y^2) - Y P_{\text{odd}}(Y^2). \quad (3)$$

2 Notation and CKKS Semantics

2.1 环与嵌入

记

$$\mathcal{S}_n = \mathbb{R}[Y]/(Y^{2n} + 1), \quad \mathcal{R}_N^{\mathbb{R}} = \mathbb{R}[X]/(X^N + 1), \quad \mathcal{R}_N^{\mathbb{Z}} = \mathbb{Z}[X]/(X^N + 1).$$

通过 $Y = X^k$ 把 $\mathcal{S}_n$ 嵌入 $\mathcal{R}_N^{\mathbb{R}}$ 。

2.2 密文语义记号

我们使用如下标准化记号来表达 CKKS 密文在 $\text{scale } \Delta$ 下所表示的消息。

定义 2.1 (消息表示关系). 若一个 CKKS 密文 ct 解密后对应的明文多项式可写成

$$m(X) = \text{Round}(\Delta F(X)) + e(X) \in \mathcal{R}_N^{\mathbb{Z}},$$

其中 $F(X) \in \mathcal{R}_N^{\mathbb{R}}$ ，而 $e(X)$ 是通常的小噪声项，则记为

$$\text{ct} \models_{\Delta} F(X).$$

该记号只关注“被表示的实系数消息” $F(X)$ ，不显式追踪噪声、RNS 分解、模链变化、rescale 细节。若实现中需要进行 level/scale 对齐，则默认按标准 CKKS 方式处理。

2.3 本文使用的同态算子

本文只用到以下四类标准算子：

(i) 自同构：

$$\text{Auto}_a(\text{ct}), \quad a \text{ 为奇数,}$$

对应

$$\sigma_a : \mathcal{R}_N^{\mathbb{R}} \rightarrow \mathcal{R}_N^{\mathbb{R}}, \quad f(X) \mapsto f(X^a).$$

(ii) 加法：

$$\text{Add}(\text{ct}_1, \text{ct}_2).$$

(iii) 减法：

$$\text{Sub}(\text{ct}_1, \text{ct}_2).$$

(iv) 明文–密文乘法：

$$\text{MulPt}_{u(X)}(\text{ct}), \quad u(X) \in \mathcal{R}_N^{\mathbb{R}}.$$

2.4 上标 $\uparrow$ 的含义

本文中的

$$\text{ct}_{\text{even}}^{\uparrow}, \quad \text{ct}_{\text{odd}, Y}^{\uparrow}, \quad \text{ct}_{\text{odd}}^{\uparrow}$$

都带有上标 $\uparrow$ 。这表示：对应消息虽然仍然存放在大环 $\mathcal{R}_N^{\mathbb{R}}$ 中，但只含 X^{2k} 的幂，因此实际上来自一个更小子环的“提升表示”。

3 Main Result

3.1 核心恒等式

由 (2) 和 (3) 立刻得到

$$P_{\text{even}}(Y^2) = \frac{P(Y) + P(-Y)}{2}, \quad (4)$$

$$Y P_{\text{odd}}(Y^2) = \frac{P(Y) - P(-Y)}{2}. \quad (5)$$

若进一步右乘 Y^{-1} ，则

$$P_{\text{odd}}(Y^2) = Y^{-1} \cdot \frac{P(Y) - P(-Y)}{2}. \quad (6)$$

在 $Y = X^k$ 的嵌入下，上式变成

$$P_{\text{even}}(X^{2k}) = \frac{P(X^k) + P(-X^k)}{2}, \quad (7)$$

$$X^k P_{\text{odd}}(X^{2k}) = \frac{P(X^k) - P(-X^k)}{2}, \quad (8)$$

$$P_{\text{odd}}(X^{2k}) = (X^k)^{-1} \cdot \frac{P(X^k) - P(-X^k)}{2}. \quad (9)$$

3.2 实现 $Y \mapsto -Y$ 的特定自同构

取

$$a = 1 + 2n. \quad (10)$$

则 a 为奇数，并且对应自同构 $\sigma_a(X) = X^a$ 。它满足

$$\sigma_a(X^k) = X^{k(1+2n)} = X^{k+N} = X^k X^N = -X^k. \quad (11)$$

因此，若 $\mathbf{ct}_P \models_{\Delta} P(X^k)$ ，则

$$\mathbf{ct}_{-} := \text{Auto}_{1+2n}(\mathbf{ct}_P)$$

满足

$$\mathbf{ct}_{-} \models_{\Delta} P(-X^k).$$

4 Correctness Theorems (No Proofs)

本节只陈述结论，不再给证明。

定理 4.1 (正向拆分：非归一化奇分支). 设

$$\mathbf{ct}_P \models_{\Delta} P(X^k).$$

定义

$$\begin{aligned} \mathbf{ct}_{-} &:= \text{Auto}_{1+2n}(\mathbf{ct}_P), \\ \mathbf{ct}_{\text{even}}^{\uparrow} &:= \text{MulPt}_{\frac{1}{2}}(\text{Add}(\mathbf{ct}_P, \mathbf{ct}_{-})), \\ \mathbf{ct}_{\text{odd}, Y}^{\uparrow} &:= \text{MulPt}_{\frac{1}{2}}(\text{Sub}(\mathbf{ct}_P, \mathbf{ct}_{-})). \end{aligned}$$

则

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}, Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k}).$$

定理 4.2 (正向拆分：归一化奇分支). 在上一结论的基础上，再定义

$$\mathbf{ct}_{\text{odd}}^{\uparrow} := \text{MulPt}_{-X^{N-k}}(\mathbf{ct}_{\text{odd}, Y}^{\uparrow}).$$

则

$$\mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k}).$$

定理 4.3 (反向重构：从非归一化奇分支). 若

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}, Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k}),$$

则

$$\text{Add}(\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd}, Y}^{\uparrow}) \models_{\Delta} P(X^k).$$

定理 4.4 (反向重构：从归一化奇分支). 若

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k}),$$

则

$$\text{Add}(\mathbf{ct}_{\text{even}}^{\uparrow}, \text{MulPt}_{X^k}(\mathbf{ct}_{\text{odd}}^{\uparrow})) \models_{\Delta} P(X^k).$$

5 Algorithms

5.1 约定

下列伪代码默认：

- 所有密文在进入某一步前都已完成必要的 `level/scale` 对齐；
- `Auto(1 + 2n)` 所需的自同构评估密钥已预生成；
- $\frac{1}{2}$ 、 X^k 、 $-X^{N-k}$ 这些明文乘子都能被构造为可参与 `plaintext-ciphertext` 乘法的明文对象。

5.2 正向拆分：保留奇分支前面的 X^k

Algorithm 1 ForwardSplitKeepY

输入：A ciphertext $\mathbf{ct}_P$ such that $\mathbf{ct}_P \models_{\Delta} P(X^k)$, where

$$P(Y) = \sum_{j=0}^{2n-1} p_j Y^j, \quad k = \frac{N}{2n}.$$

输出：Two ciphertexts $\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd},Y}^{\uparrow}$ such that

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd},Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k}).$$

- 1: $a \leftarrow 1 + 2n$
 - 2: $\mathbf{ct}_{-} \leftarrow \text{Auto}_a(\mathbf{ct}_P)$ $\triangleright \mathbf{ct}_{-} \models_{\Delta} P(-X^k)$
 - 3: $\mathbf{s}_{+} \leftarrow \text{Add}(\mathbf{ct}_P, \mathbf{ct}_{-})$
 - 4: $\mathbf{s}_{-} \leftarrow \text{Sub}(\mathbf{ct}_P, \mathbf{ct}_{-})$
 - 5: $\mathbf{ct}_{\text{even}}^{\uparrow} \leftarrow \text{MulPt}_{\frac{1}{2}}(\mathbf{s}_{+})$ $\triangleright \mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k})$
 - 6: $\mathbf{ct}_{\text{odd},Y}^{\uparrow} \leftarrow \text{MulPt}_{\frac{1}{2}}(\mathbf{s}_{-})$ $\triangleright \mathbf{ct}_{\text{odd},Y}^{\uparrow} \models_{\Delta} X^k P_{\text{odd}}(X^{2k})$
 - 7: **return** $(\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd},Y}^{\uparrow})$
-

5.3 正向拆分：去掉奇分支前面的 X^k

Algorithm 2 ForwardSplitNormalized

输入：A ciphertext $\mathbf{ct}_P$ such that $\mathbf{ct}_P \models_{\Delta} P(X^k)$.

输出：Two ciphertexts $\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd}}^{\uparrow}$ such that

$$\mathbf{ct}_{\text{even}}^{\uparrow} \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k}).$$

- 1: $(\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd},Y}^{\uparrow}) \leftarrow \text{FORWARDSPPLITKEEPY}(\mathbf{ct}_P)$
 - 2: $u(X) \leftarrow -X^{N-k}$ $\triangleright$ Since $(X^k)^{-1} = -X^{N-k}$
 - 3: $\mathbf{ct}_{\text{odd}}^{\uparrow} \leftarrow \text{MulPt}_{u(X)}(\mathbf{ct}_{\text{odd},Y}^{\uparrow})$ $\triangleright \mathbf{ct}_{\text{odd}}^{\uparrow} \models_{\Delta} P_{\text{odd}}(X^{2k})$
 - 4: **return** $(\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd}}^{\uparrow})$
-

5.4 反向重构：输入保留 X^k 的奇分支

Algorithm 3 MergeFromKeepY

输入: Two ciphertexts $\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd},Y}^\uparrow$ such that

$$\mathbf{ct}_{\text{even}}^\uparrow \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd},Y}^\uparrow \models_{\Delta} X^k P_{\text{odd}}(X^{2k}).$$

输出: A ciphertext $\mathbf{ct}_P$ such that

$$\mathbf{ct}_P \models_{\Delta} P(X^k).$$

1: $\mathbf{ct}_P \leftarrow \text{Add}(\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd},Y}^\uparrow)$

2: **return** $\mathbf{ct}_P$

5.5 反向重构：输入为归一化奇分支

Algorithm 4 MergeFromNormalized

输入: Two ciphertexts $\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}}^\uparrow$ such that

$$\mathbf{ct}_{\text{even}}^\uparrow \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}}^\uparrow \models_{\Delta} P_{\text{odd}}(X^{2k}).$$

输出: A ciphertext $\mathbf{ct}_P$ such that

$$\mathbf{ct}_P \models_{\Delta} P(X^k).$$

1: $\mathbf{t} \leftarrow \text{MulPt}_{X^k}(\mathbf{ct}_{\text{odd}}^\uparrow)$

$$\triangleright \mathbf{t} \models_{\Delta} X^k P_{\text{odd}}(X^{2k})$$

2: $\mathbf{ct}_P \leftarrow \text{Add}(\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{t})$

3: **return** $\mathbf{ct}_P$

5.6 一个最小执行模板

如果实现中需要明确地把“生成自同构密钥”也写入流程，那么最小模板可写成：

Algorithm 5 EndToEndSplitAndMerge

输入: A ciphertext $\mathbf{ct}_P \models_{\Delta} P(X^k)$; secret-dependent evaluation-key generation is available offline.

输出: A reconstructed ciphertext $\widehat{\mathbf{ct}}_P \models_{\Delta} P(X^k)$.

1: $a \leftarrow 1 + 2n$

2: Generate / load the automorphism key for Auto_a

3: $(\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}}^\uparrow) \leftarrow \text{FORWARDSPITNORMALIZED}(\mathbf{ct}_P)$

4: $\widehat{\mathbf{ct}}_P \leftarrow \text{MERGEFROMNORMALIZED}(\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}}^\uparrow)$

5: **return** $\widehat{\mathbf{ct}}_P$

6 Implementation Notes

6.1 总则

虽然本文给出的算法在代数层面非常简洁，但把它落到具体库时，至少要注意四件事：

- (1) **自同构索引的表示方式**：数学上我们写的是 σ_{1+2n} ，即

$$X \mapsto X^{1+2n}.$$

但不同库对 Galois 元素、rotation index、automorphism index 的表示方式不同。实现时必须确认：你传入的“索引 / Galois element”确实实现了

$$X^k \longmapsto -X^k.$$

- (2) **plaintext 的构造方式**：本文用到的明文乘子是

$$\frac{1}{2}, \quad X^k, \quad -X^{N-k}.$$

常数 $1/2$ 通常最容易；而单项式 X^k 、 $-X^{N-k}$ 往往需要底层的 plaintext-polynomial 构造路径。如果库的高层 CKKS 接口只暴露“槽编码器”，那么实现这一步可能需要转到更底层的 plaintext 对象或自定义构造函数。

- (3) **level/scale 对齐**：做

$$\mathbf{ct}_P \pm \mathbf{ct}_-$$

之前，必须保证二者在同一 level、同一 scale（或者能通过库的自动机制对齐）。做 plaintext multiplication 之后，是否需要 rescale，取决于库的语义和你给 plaintext 指定的 scale。

- (4) **系数域语义与槽语义**：本文算法在“多项式消息语义”下描述。如果库层面默认以槽语义暴露 CKKS plaintext/ciphertext，那么你需要清楚自己是在使用高层编码接口，还是在直接操纵环里的 plaintext polynomial。

6.2 OpenFHE

在 OpenFHE 中，可将本文四类抽象操作大致映射为：

抽象操作	OpenFHE 中的对应接口
生成 automorphism key	<code>EvalAutomorphismKeyGen(privateKey, indexList)</code>
应用 automorphism	<code>EvalAutomorphism(ciphertext, i, evalKeyMap)</code>
密文加法 / 减法	<code>EvalAdd, EvalSub</code>
明文-密文乘法	<code>EvalMult(ciphertext, scalar)</code> 或 <code>EvalMult(ciphertext, plaintext)</code>

建议写法 若你已经知道目标 automorphism index 对应数学上的 σ_{1+2n} ，那么正向拆分最接近的实现骨架是：

```
ct_ ← cc->EvalAutomorphism(ct_P, a, evalAutoKeys),
ct_even^↑ ← cc->EvalMult(cc->EvalAdd(ct_P, ct_), 0.5),
ct_odd,Y^↑ ← cc->EvalMult(cc->EvalSub(ct_P, ct_), 0.5).
```

注意 OpenFHE 的高层 CKKS 工作流通常以 packed plaintext 为中心。如果你要真正实现

$$\text{MulPt}_{X^k}(\cdot), \quad \text{MulPt}_{-X^{N-k}}(\cdot),$$

需要确认你所用版本是否方便构造“系数域明文多项式”作为 Plaintext；否则更稳妥的办法是自己在较低层构造该明文对象，再交给 EvalMult。

6.3 Microsoft SEAL

在 SEAL 中，本文抽象操作可大致映射为：

抽象操作	Microsoft SEAL 中的对应接口
生成 Galois keys	KeyGenerator::create_galois_keys(...)
应用 automorphism	Evaluator::apply_galois_inplace(...)
密文加法 / 减法	Evaluator::add_inplace, Evaluator::sub_inplace
明文-密文乘法	Evaluator::multiply_plain_inplace(...)
plaintext 预处理	Evaluator::transform_to_ntt_inplace(...) (若需要)

建议写法 SEAL 中通常会把 ct_P 复制到临时对象，再原地修改。例如：

```
ct_ ← ct_P;
evaluator.apply_galois_inplace(ct_, galois_elt, galois_keys);
ct_even^↑ ← ct_P;
evaluator.add_inplace(ct_even^↑, ct_);
evaluator.multiply_plain_inplace(ct_even^↑, plain_half);
ct_odd,Y^↑ ← ct_P;
evaluator.sub_inplace(ct_odd,Y^↑, ct_);
evaluator.multiply_plain_inplace(ct_odd,Y^↑, plain_half).
```

注意 SEAL 对 CKKS plaintext 的 NTT 形式、parms_id、scale 对齐非常严格。尤其是在做

multiply_plain_inplace

前，往往需要确保 plaintext 已经处于正确的 level，并在必要时通过

`mod_switch_to_inplace,      transform_to_ntt_inplace`

进行预处理。若要实现单项式 X^k 或 $-X^{N-k}$ 的明文乘法，通常也需要手工构造对应的 `Plaintext`。

6.4 Lattigo v5

Lattigo v5 把 automorphism 更明确地暴露在 RLWE 层，而 CKKS 层的 evaluator 负责常规算术。可以大致映射为：

抽象操作	Lattigo v5 中的对应接口
生成 Galois key	<code>rlwe.KeyGenerator.GenGaloisKey / GenGaloisKeyNew</code>
应用 automorphism	<code>rlwe.Evaluator.Automorphism</code>
获取某些标准 Galois 元素	<code>ckks.Parameters.GaloisElementForRotation(...)</code>
密文加法 / 减法	<code>ckks.Evaluator.Add, ckks.Evaluator.Sub</code>
明文-密文乘法	<code>ckks.Evaluator.Mul</code>
rescale	<code>ckks.Evaluator.Rescale</code>

建议写法 如果你已知目标 Galois element `galEl` 实现了

$$X^k \mapsto -X^k,$$

那么可在 RLWE 层先做 automorphism，再回到 CKKS evaluator 做算术。一个自然的分层写法是：

- (1) 用 `rlwe.KeyGenerator.GenGaloisKeyNew(galEl, sk, ...)` 生成所需 Galois key；
- (2) 用 `rlwe.Evaluator.Automorphism(ctIn, galEl, ctOut)` 得到 `ct_`；
- (3) 用 `ckks.Evaluator.Add/Sub/Mul` 完成

$$\mathbf{ct}_{\text{even}}^{\uparrow}, \mathbf{ct}_{\text{odd}, Y}^{\uparrow}, \mathbf{ct}_{\text{odd}}^{\uparrow}$$

的构造；

- (4) 若 plaintext multiplication 改变了 scale，则按需要调用 `Rescale`。

注意 Lattigo 的 `GaloisElementForRotation(k)` 是“标准槽旋转”的 helper；而本文需要的是“在多项式环中实现 $X^k \mapsto -X^k$ 的 automorphism”。这两者不应混淆。若你不能把目标 automorphism 等价地转化为一个标准 rotation，那么更稳妥的做法是直接在 RLWE 层使用所需的 `galEl`。

7 Practical Checklist

下面给出一个真正实现时的检查清单。

C1. 确认 automorphism: 你的库里传入的 index / Galois element 是否真的实现了

$$X^k \mapsto -X^k?$$

C2. 确认 plaintext 构造: 是否能构造

$$\frac{1}{2}, \quad X^k, \quad -X^{N-k}$$

这三个明文乘子?

C3. 确认 level 对齐: $\mathbf{ct}_P$ 与 $\mathbf{ct}_-$ 在进入 Add / Sub 前是否处于同一 level?

C4. 确认 scale 对齐: $\mathbf{ct}_P$ 与 $\mathbf{ct}_-$ 的 scale 是否一致? 明文乘法后是否需要 rescale?

C5. 确认密文语义: 你当前实现处理的是“槽语义 CKKS 流程”还是“本文采用的多项式语义流程”? 不要把两者混用。

8 Conclusion

本文第二版不再重复第一版的推导, 而是将最终可用结论整理成了以下四个算法:

(1) FORWARD_SPLIT_KEEPY:

$$\mathbf{ct}_P \mapsto (\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}, Y}^\uparrow), \quad \mathbf{ct}_{\text{even}}^\uparrow \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}, Y}^\uparrow \models_{\Delta} X^k P_{\text{odd}}(X^{2k});$$

(2) FORWARD_SPLIT_NORMALIZED:

$$\mathbf{ct}_P \mapsto (\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}}^\uparrow), \quad \mathbf{ct}_{\text{even}}^\uparrow \models_{\Delta} P_{\text{even}}(X^{2k}), \quad \mathbf{ct}_{\text{odd}}^\uparrow \models_{\Delta} P_{\text{odd}}(X^{2k});$$

(3) MERGE_FROM_KEEPY:

$$(\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}, Y}^\uparrow) \mapsto \mathbf{ct}_P;$$

(4) MERGE_FROM_NORMALIZED:

$$(\mathbf{ct}_{\text{even}}^\uparrow, \mathbf{ct}_{\text{odd}}^\uparrow) \mapsto \mathbf{ct}_P.$$

就抽象结构而言, 这四个算法已经是完整的; 实现时只需把

$$\text{Auto}_{1+2n}, \text{Add}, \text{Sub}, \text{MulPt}.$$

分别映射到具体库的 API, 并处理好 automorphism key、plaintext 构造以及 level/scale 对齐即可。